

Predicting Soil Organic Carbon from AlphaEarth Embeddings

STAT 4000/5000 Final Project — Code Notebook

Luke Robinson

2026-04-24

Contents

1	Setup	1
2	Method 1: Exploratory Data Analysis	3
2.1	Distribution of SOC	3
2.2	SOC by land cover	4
2.3	Correlation between AlphaEarth dimensions and log-SOC	4
3	Method 2: Hypothesis Testing	7
3.1	Assumption checks	8
3.2	Welch's two-sample t -test	9
4	Method 3: Bootstrap Confidence Intervals	9
4.1	Bootstrap CI for the strongest correlation	10
4.2	Bootstrap CI for the group mean difference	12
5	Beyond the three: PCA and Random Forest	13
5.1	Principal component analysis	13
5.2	Spatial-block cross-validated Random Forest	14
6	Session info	17

1 Setup

```
library(tidyverse)
library(boot)
library(corrplot)
library(car)
library(ranger)
```

```

dir.create("figures", showWarnings = FALSE)

df <- read.csv("soil_samples.csv")
ae_cols <- sprintf("A%02d", 0:63)
cat("Rows:", nrow(df), " Cols:", ncol(df), "\n")

```

```
## Rows: 3000 Cols: 68
```

soil_samples.csv was extracted from Google Earth Engine (01_extract_data.py). Each row is a random point in the contiguous US with 64 AlphaEarth Foundations 2024 annual embedding bands (A00..A63), OpenLandMap Soil Organic Carbon at 0 cm (soc, g/kg), MODIS IGBP land cover (land_cover), and lon/lat.

```

# IGBP land-cover labels
igbp_labels <- c(
  "1" = "Evergreen Needleleaf Forest",
  "2" = "Evergreen Broadleaf Forest",
  "3" = "Deciduous Needleleaf Forest",
  "4" = "Deciduous Broadleaf Forest",
  "5" = "Mixed Forest",
  "6" = "Closed Shrublands",
  "7" = "Open Shrublands",
  "8" = "Woody Savannas",
  "9" = "Savannas",
  "10" = "Grasslands",
  "11" = "Permanent Wetlands",
  "12" = "Croplands",
  "13" = "Urban and Built-up",
  "14" = "Cropland/Natural Vegetation Mosaic",
  "15" = "Permanent Snow and Ice",
  "16" = "Barren",
  "17" = "Water Bodies"
)

df <- df %>%
  mutate(
    land_cover = as.character(land_cover),
    lc_label   = igbp_labels[land_cover],
    log_soc    = log1p(soc)
  )

# basic numeric checks
stopifnot(all(!is.na(df$soc)))
range(df$soc); median(df$soc)

```

```
## [1] 0 41
```

```
## [1] 3
```

2 Method 1: Exploratory Data Analysis

We start with **exploratory data analysis** (numerical and graphical summaries of the response variable, the predictors, and their bivariate relationships).

2.1 Distribution of SOC

```
p1 <- ggplot(df, aes(soc)) +  
  geom_histogram(bins = 40, fill = "#5b8def", color = "white") +  
  labs(x = "SOC (g/kg)", y = "Count", title = "Raw SOC") +  
  theme_minimal()  
  
p2 <- ggplot(df, aes(log_soc)) +  
  geom_histogram(bins = 40, fill = "#27ae60", color = "white") +  
  labs(x = "log(1 + SOC)", y = "Count", title = "log-transformed SOC") +  
  theme_minimal()  
  
gridExtra::grid.arrange(p1, p2, ncol = 2)
```

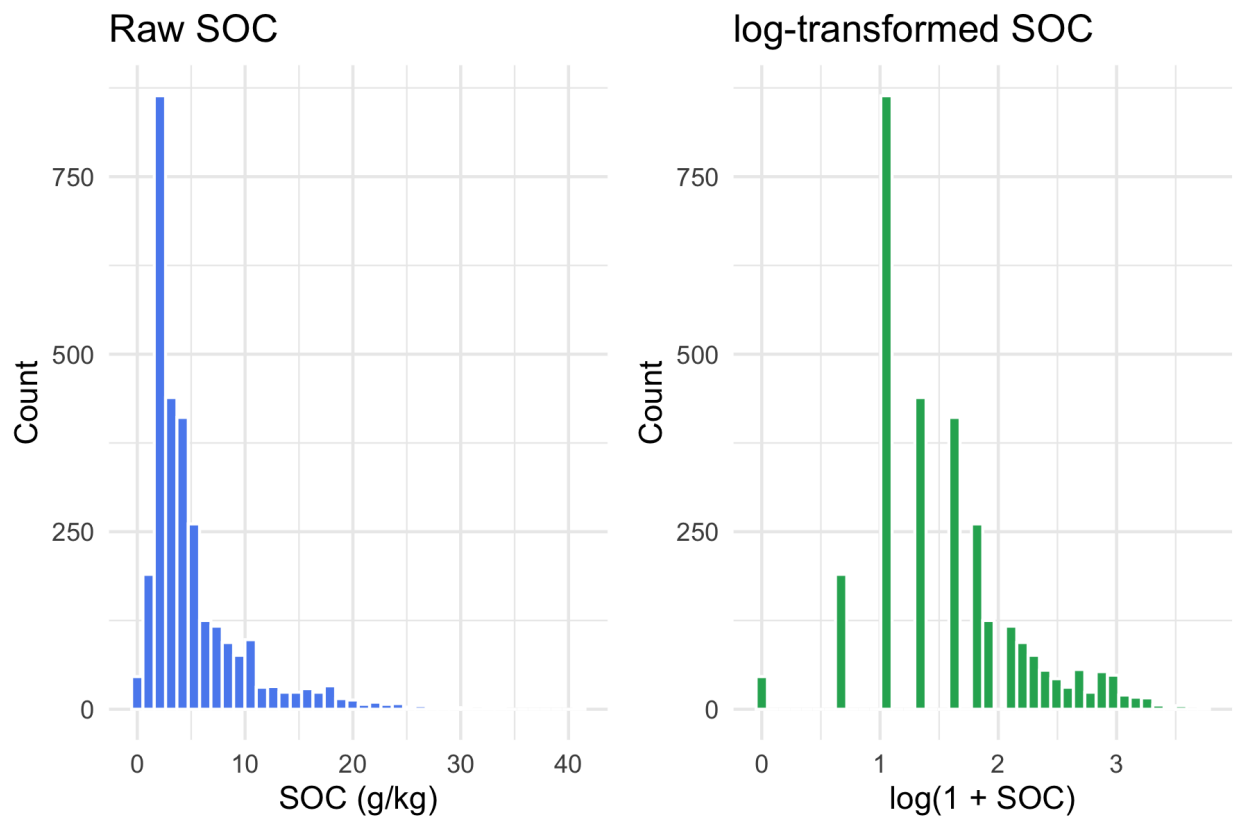


Figure 1: Distribution of soil organic carbon. Raw values are right-skewed; the log1p transform is approximately symmetric.

```
soc_summary <- df %>%
  summarise(
    n      = n(),
    mean   = mean(soc),
    sd     = sd(soc),
    median = median(soc),
    iqr    = IQR(soc),
    skewness = mean((soc - mean(soc))^3) / sd(soc)^3
  )
soc_summary
```

```
##      n      mean      sd median iqr skewness
## 1 3000 5.023333 4.800952      3    4 2.391268
```

The right-skew (2.39) violates approximate normality. We use $\log_{1p}(\text{SOC})$ for downstream inferential analyses.

2.2 SOC by land cover

```
top_lc <- df %>% count(lc_label, sort = TRUE) %>% slice_head(n = 8) %>% pull(lc_label)

df %>%
  filter(lc_label %in% top_lc) %>%
  mutate(lc_label = fct_reorder(lc_label, log_soc, median)) %>%
  ggplot(aes(lc_label, log_soc, fill = lc_label)) +
  geom_boxplot(outlier.alpha = 0.4) +
  coord_flip() +
  labs(x = NULL, y = "log(1 + SOC)") +
  theme_minimal() +
  theme(legend.position = "none")
```

2.3 Correlation between AlphaEarth dimensions and log-SOC

We examine the marginal Pearson correlation between each of the 64 embedding dimensions and $\log(1 + \text{SOC})$.

```
cors <- sapply(ae_cols, function(c) cor(df[[c]], df$log_soc))
cor_df <- tibble(dim = ae_cols, r = cors) %>%
  arrange(desc(abs(r)))

ggplot(cor_df, aes(reorder(dim, r), r, fill = r > 0)) +
  geom_col() +
  scale_fill_manual(values = c("#c0392b", "#2980b9"), guide = "none") +
  labs(x = NULL, y = "Pearson r vs. log-SOC") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 90, size = 5))
```

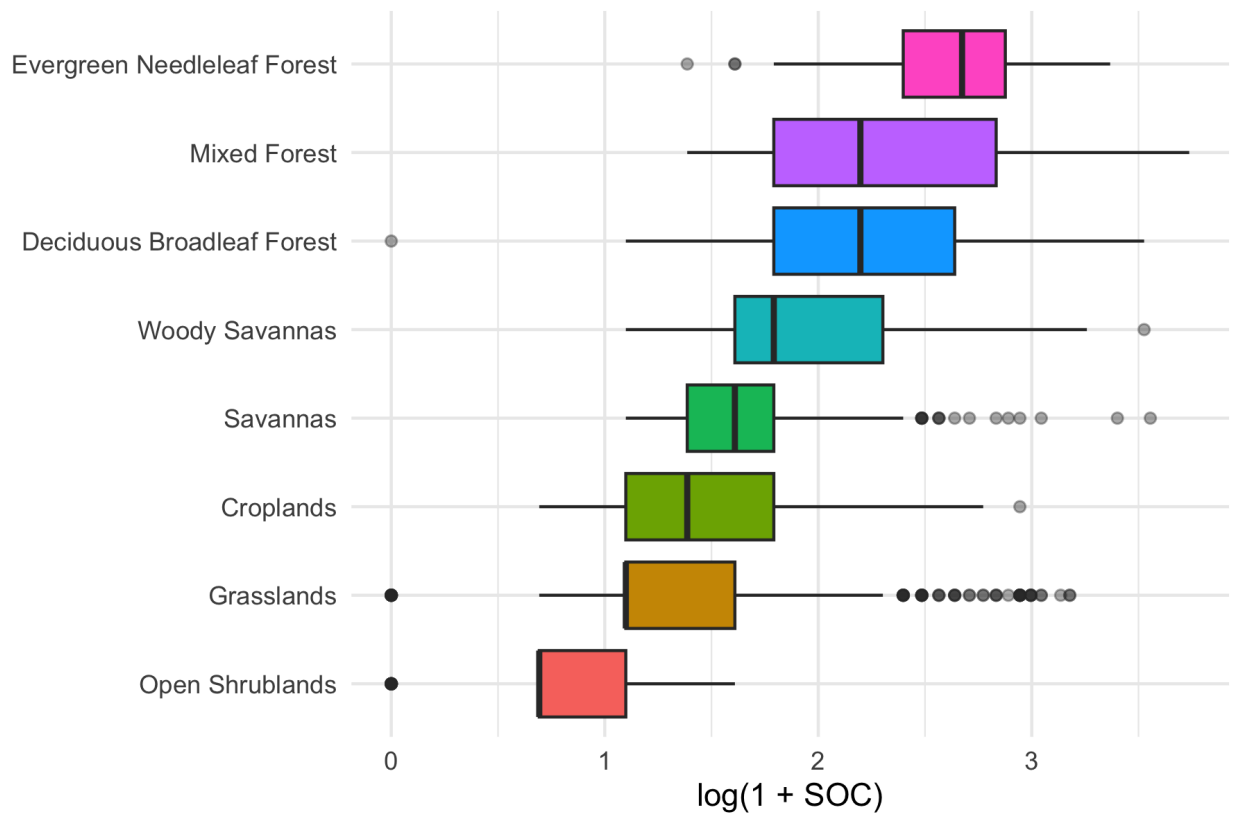


Figure 2: log-SOC by IGBP land-cover class. Forest classes have the highest median; barren and shrubland the lowest.

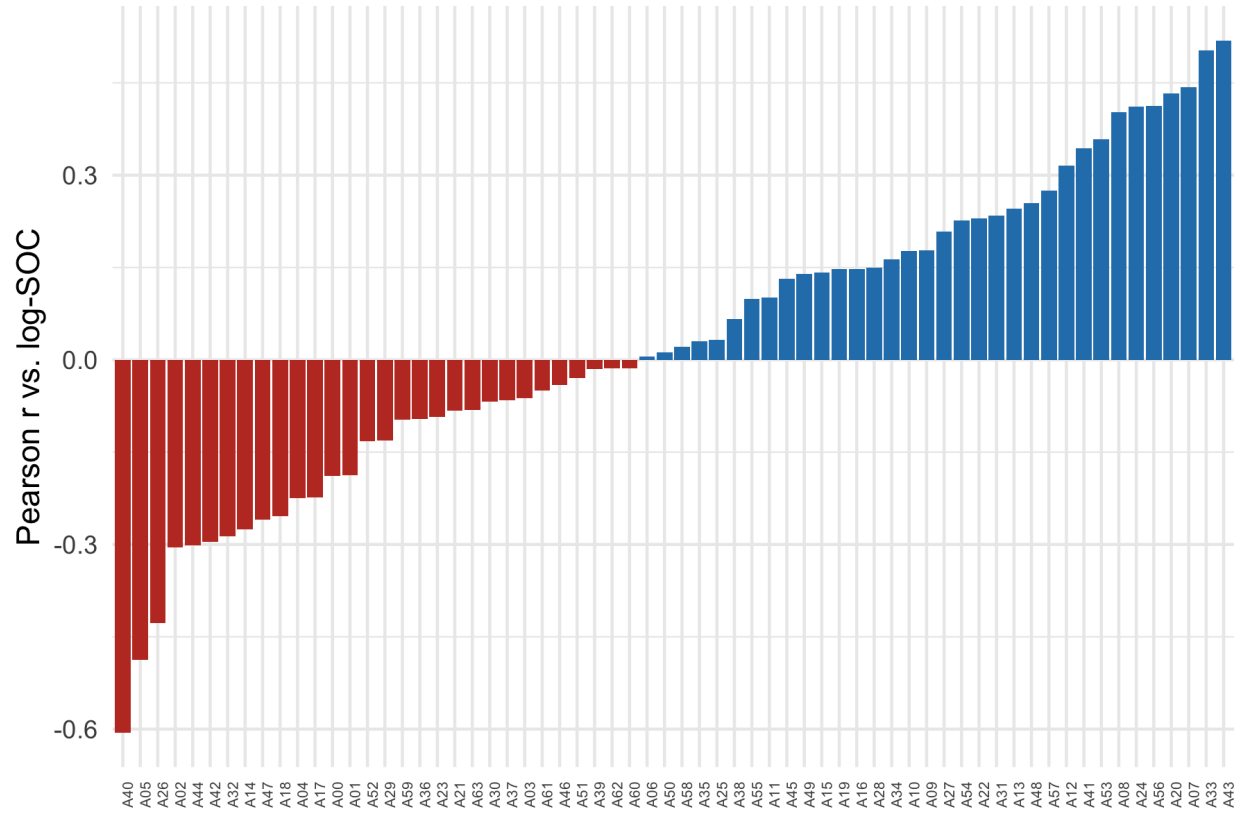


Figure 3: Pearson correlation between each of the 64 AlphaEarth dimensions and log-SOC. Several dimensions exceed $|r| = 0.3$.

```

top_dim <- cor_df$dim[1]
top_r    <- cor_df$r[1]
cat("Top dimension:", top_dim, " r =", round(top_r, 3), "\n")

```

```
## Top dimension: A40    r = -0.606
```

```

top4 <- cor_df$dim[1:4]
df_long <- df %>%
  pivot_longer(all_of(top4), names_to = "dim", values_to = "value")

ggplot(df_long, aes(value, log_soc)) +
  geom_point(alpha = 0.25, size = 0.6, color = "#34495e") +
  geom_smooth(method = "loess", se = FALSE, color = "#e67e22") +
  facet_wrap(~ dim, scales = "free_x") +
  labs(x = "AlphaEarth value", y = "log(1 + SOC)") +
  theme_minimal()

```

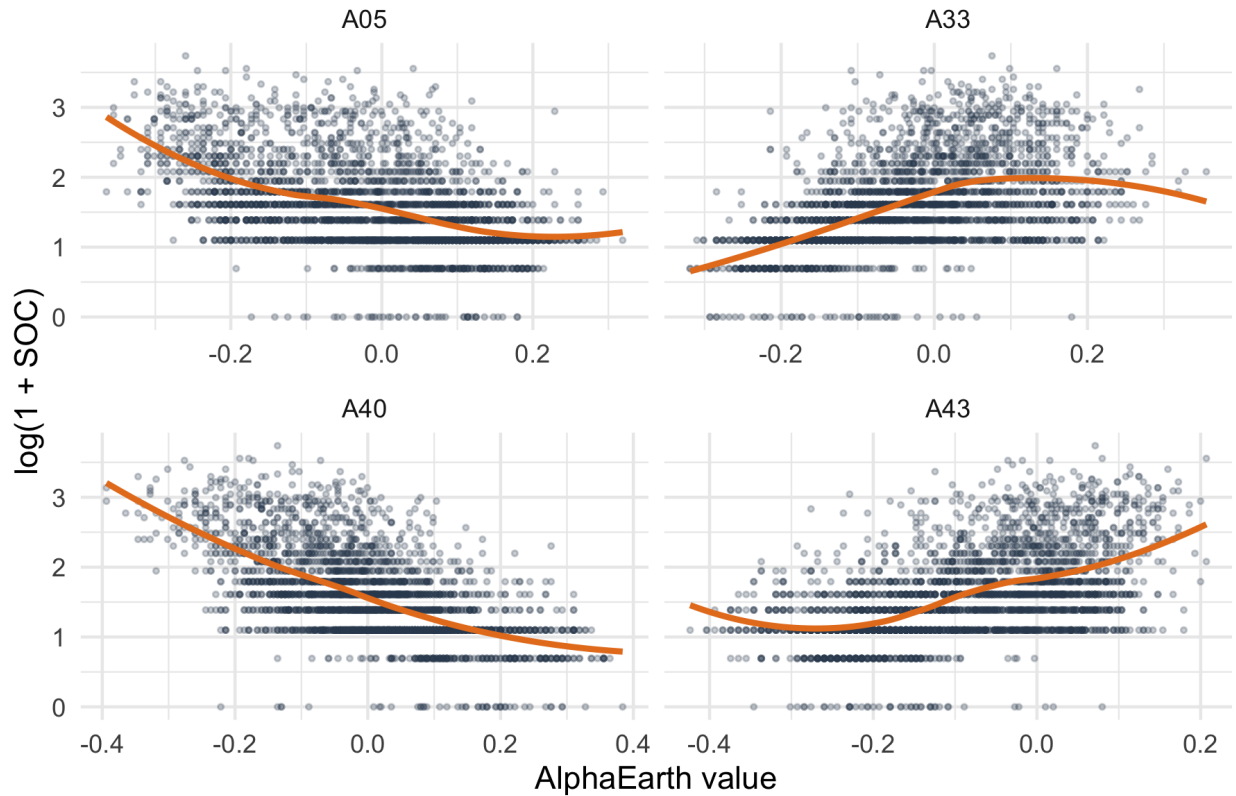


Figure 4: Scatter of log-SOC against the four AlphaEarth dimensions with the largest $|r|$.

3 Method 2: Hypothesis Testing

We test whether mean log-SOC differs between the two largest IGBP groups in our sample, **Grasslands** (class 10) and **Croplands** (class 12). A two-sample Welch's t -test is appropriate because (a) the groups are

independent, (b) the response is approximately symmetric on the log scale, and (c) we do not assume equal variances.

```
two_groups <- df %>% filter(land_cover %in% c("10", "12"))
tab <- two_groups %>% count(lc_label)
tab
```

```
##      lc_label      n
## 1  Croplands    543
## 2  Grasslands  1052
```

3.1 Assumption checks

```
par(mfrow = c(1, 2))
qqnorm(two_groups$log_soc[two_groups$land_cover == "10"],
       main = "Grasslands"); qqline(two_groups$log_soc[two_groups$land_cover == "10"])
qqnorm(two_groups$log_soc[two_groups$land_cover == "12"],
       main = "Croplands"); qqline(two_groups$log_soc[two_groups$land_cover == "12"])
```

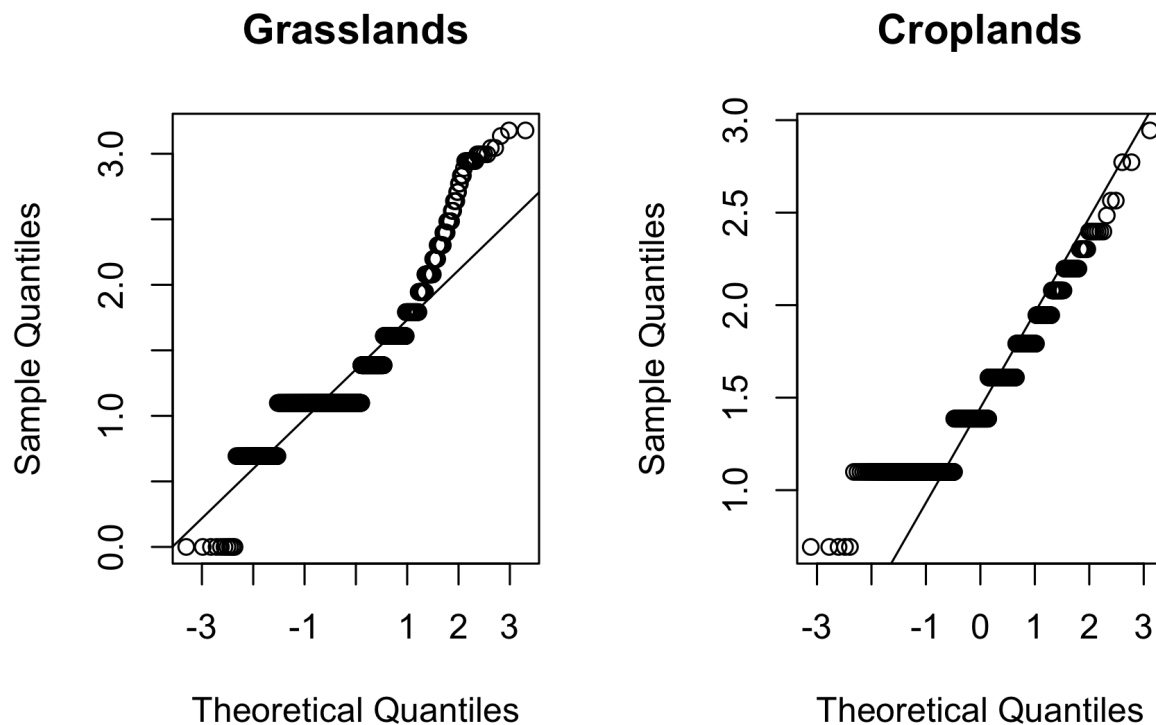


Figure 5: QQ plots of log-SOC for the two groups. Both are close to linear, supporting an approximately normal sampling distribution of the mean (large n further by the CLT).


```
par(mfrow = c(1, 1))

# Levene's test for equal variances
levene_res <- car::leveneTest(log_soc ~ factor(land_cover), data = two_groups)
levene_res
```

```
## Levene's Test for Homogeneity of Variance (center = median)
##           Df F value Pr(>F)
## group      1  2.2635 0.1327
##           1593
```

Levene's test gives $p = 0.133$; with unequal variances likely, Welch's t is the right choice.

3.2 Welch's two-sample t -test

```
t_res <- t.test(log_soc ~ land_cover, data = two_groups,
               var.equal = FALSE, conf.level = 0.95)
t_res
```

```
##
## Welch Two Sample t-test
##
## data: log_soc by land_cover
## t = -6.2357, df = 1326, p-value = 6.033e-10
## alternative hypothesis: true difference in means between group 10 and group 12 is not equal to 0
## 95 percent confidence interval:
## -0.17623918 -0.09188705
## sample estimates:
## mean in group 10 mean in group 12
##           1.356056           1.490119
```

```
# Cohen's d (using pooled sd for effect size)
g1 <- two_groups$log_soc[two_groups$land_cover == "10"]
g2 <- two_groups$log_soc[two_groups$land_cover == "12"]
pooled_sd <- sqrt(((length(g1)-1)*var(g1) + (length(g2)-1)*var(g2)) /
                  (length(g1)+length(g2)-2))
cohens_d <- (mean(g1) - mean(g2)) / pooled_sd
cat("Cohen's d =", round(cohens_d, 3), "\n")
```

```
## Cohen's d = -0.307
```

4 Method 3: Bootstrap Confidence Intervals

We use the **nonparametric bootstrap** ($B = 10,000$ resamples, BCa intervals) to obtain confidence intervals for two quantities without distributional assumptions: (i) the Pearson correlation between the strongest AlphaEarth dimension and log-SOC, and (ii) the same group mean difference tested above.

4.1 Bootstrap CI for the strongest correlation

```
B <- 10000
top_vec <- df[[top_dim]]
y_vec <- df$log_soc

corr_stat <- function(d, idx) cor(d[idx, 1], d[idx, 2])

set.seed(42)
boot_cor <- boot(data.frame(x = top_vec, y = y_vec), corr_stat, R = B)
boot_cor_ci <- boot.ci(boot_cor, type = "bca")
boot_cor

##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = data.frame(x = top_vec, y = y_vec), statistic = corr_stat,
##      R = B)
##
##
## Bootstrap Statistics :
##      original      bias      std. error
## t1* -0.6055629  8.543581e-06  0.01205585

boot_cor_ci

## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 10000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_cor, type = "bca")
##
## Intervals :
## Level      BCa
## 95%      (-0.6284, -0.5812 )
## Calculations and Intervals on Original Scale

ci <- boot_cor_ci$bca[4:5]
ggplot(data.frame(r = boot_cor$t[, 1]), aes(r)) +
  geom_histogram(bins = 60, fill = "#5b8def", color = "white") +
  geom_vline(xintercept = boot_cor$t0, color = "black", linewidth = 1) +
  geom_vline(xintercept = ci, color = "#c0392b", linetype = "dashed") +
  labs(x = paste0("Bootstrap r (", top_dim, " vs. log-SOC)",
    y = "Frequency") +
  theme_minimal()
```

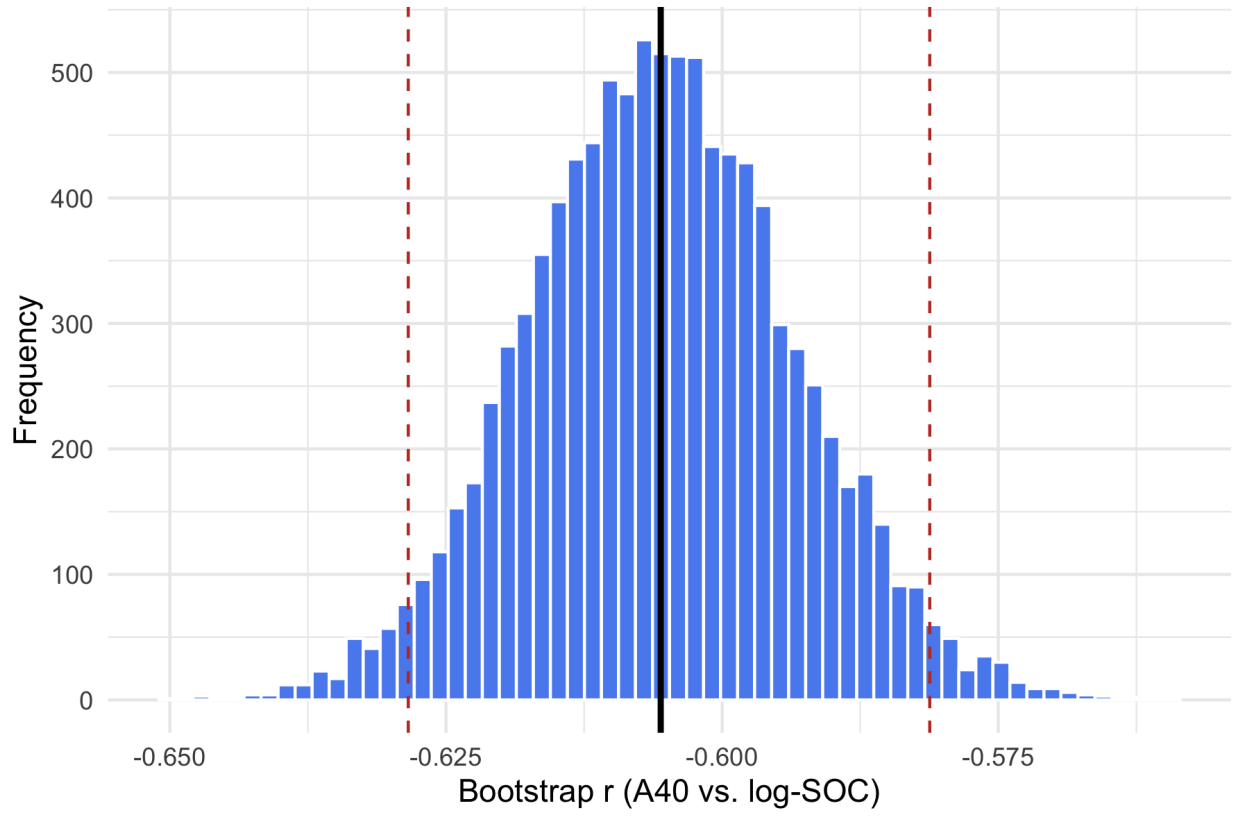


Figure 6: Bootstrap distribution ($B=10,000$) of the Pearson correlation between the strongest AlphaEarth dimension and log-SOC. The BCa 95% CI is shown as red dashed lines.

4.2 Bootstrap CI for the group mean difference

```
mean_diff <- function(d, idx) {
  s <- d[idx, ]
  mean(s$log_soc[s$land_cover == "10"]) -
    mean(s$log_soc[s$land_cover == "12"])
}

set.seed(42)
boot_md <- boot(two_groups, mean_diff, R = B,
               strata = factor(two_groups$land_cover))
boot_md_ci <- boot.ci(boot_md, type = "bca")
boot_md

##
## STRATIFIED BOOTSTRAP
##
##
## Call:
## boot(data = two_groups, statistic = mean_diff, R = B, strata = factor(two_groups$land_cover))
##
##
## Bootstrap Statistics :
##      original      bias      std. error
## t1*  -0.1340631 -0.0002818041  0.02130558

boot_md_ci

## BOOTSTRAP CONFIDENCE INTERVAL CALCULATIONS
## Based on 10000 bootstrap replicates
##
## CALL :
## boot.ci(boot.out = boot_md, type = "bca")
##
## Intervals :
## Level      BCa
## 95%      (-0.1746, -0.0914 )
## Calculations and Intervals on Original Scale

cat("\nFor comparison, Welch's t-test 95% CI:",
    signif(t_res$conf.int[1], 3), "to", signif(t_res$conf.int[2], 3), "\n")

##
## For comparison, Welch's t-test 95% CI: -0.176 to -0.0919

ci_md <- boot_md_ci$bca[4:5]
ggplot(data.frame(d = boot_md$t[, 1]), aes(d)) +
  geom_histogram(bins = 60, fill = "#27ae60", color = "white") +
  geom_vline(xintercept = boot_md$t0, color = "black", linewidth = 1) +
  geom_vline(xintercept = ci_md, color = "#c0392b", linetype = "dashed") +
  geom_vline(xintercept = 0, color = "grey50", linetype = "dotted") +
```

```
labs(x = "Bootstrap mean difference (Grasslands - Croplands)", y = "Frequency") +
theme_minimal()
```

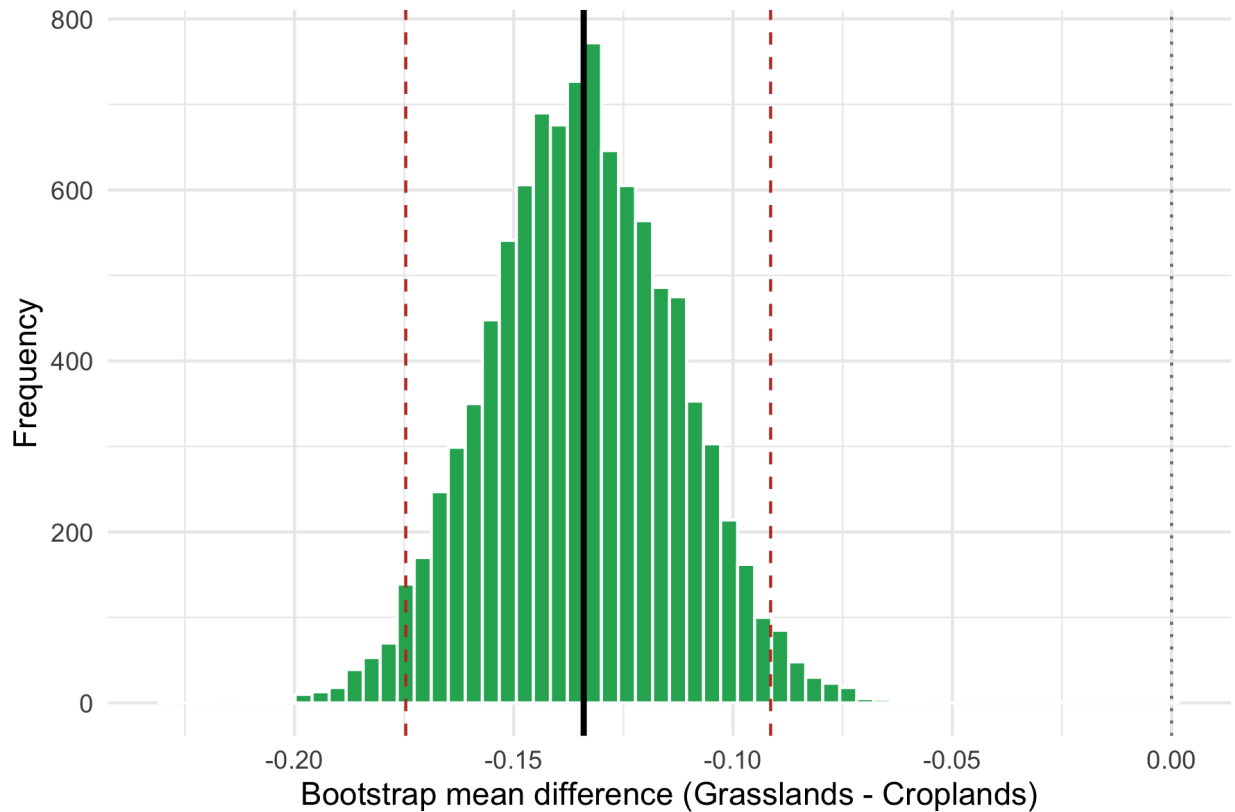


Figure 7: Bootstrap distribution ($B=10,000$) of the Grasslands – Croplands log-SOC mean difference, with stratified resampling. BCa 95% CI in red.

5 Beyond the three: PCA and Random Forest

These analyses are *outside* the three class techniques but directly answer the proposal’s research question — can the AlphaEarth latent space predict SOC? — and are reported as a supplementary analysis.

5.1 Principal component analysis

```
pca <- prcomp(df[, ae_cols], center = TRUE, scale. = FALSE)
varexp <- pca$sdev^2 / sum(pca$sdev^2)
cum <- cumsum(varexp)

screes <- tibble(component = seq_along(varexp), cum_var = cum)
ggplot(screes, aes(component, cum_var)) +
  geom_line(color = "#5b8def", linewidth = 1) +
  geom_point(color = "#5b8def") +
  geom_hline(yintercept = 0.9, linetype = "dashed", color = "grey50") +
```

```
labs(x = "Principal component", y = "Cumulative variance explained") +
theme_minimal()
```

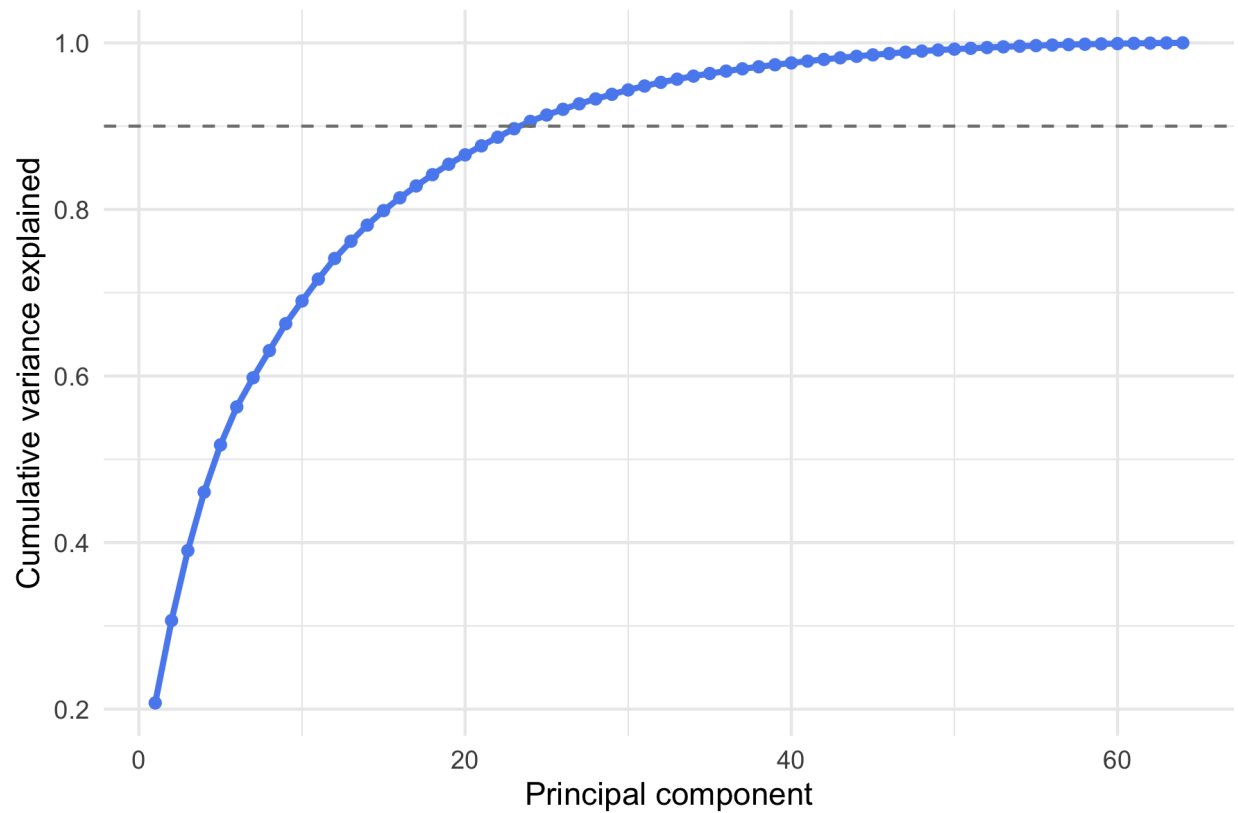


Figure 8: Cumulative variance explained by AlphaEarth principal components.

```
cat("PCs to explain 90% variance:", which(cum >= 0.90)[1], "\n")
```

```
## PCs to explain 90% variance: 24
```

```
cat("PCs to explain 95% variance:", which(cum >= 0.95)[1], "\n")
```

```
## PCs to explain 95% variance: 32
```

5.2 Spatial-block cross-validated Random Forest

We assess predictive skill with 5-fold spatial cross-validation, splitting on 1° latitude/longitude blocks so neighbouring points cannot leak between train and test.

```
df$block <- paste0(round(df$lon), "_", round(df$lat))
unique_blocks <- unique(df$block)
set.seed(42)
fold_assign <- sample(seq_len(5), length(unique_blocks), replace = TRUE)
df$fold <- fold_assign[match(df$block, unique_blocks)]
```

```

rf_form <- as.formula(
  paste("log_soc ~", paste(ae_cols, collapse = " + "))
)

cv_res <- map_dfr(1:5, function(k) {
  train <- df %>% filter(fold != k)
  test  <- df %>% filter(fold == k)
  m <- ranger(rf_form, data = train, num.trees = 500, seed = 42)
  pred <- predict(m, data = test)$predictions
  tibble(
    fold = k,
    n_test = nrow(test),
    rmse = sqrt(mean((pred - test$log_soc)^2)),
    r2 = cor(pred, test$log_soc)^2
  )
})
cv_res

```

```

## # A tibble: 5 x 4
##   fold n_test rmse   r2
##   <int> <int> <dbl> <dbl>
## 1     1     628 0.327 0.731
## 2     2     651 0.323 0.763
## 3     3     581 0.329 0.730
## 4     4     567 0.314 0.760
## 5     5     573 0.309 0.755

```

```

cat("Mean R^2:", round(mean(cv_res$r2), 3),
    " Mean RMSE:", round(mean(cv_res$rmse), 3), "\n")

```

```

## Mean R^2: 0.748   Mean RMSE: 0.32

```

```

m_full <- ranger(rf_form, data = df, num.trees = 500,
  importance = "impurity", seed = 42)
imp <- enframe(m_full$variable.importance, "dim", "importance") %>%
  arrange(desc(importance)) %>% slice_head(n = 15)

ggplot(imp, aes(reorder(dim, importance), importance)) +
  geom_col(fill = "#34495e") +
  coord_flip() +
  labs(x = NULL, y = "Random Forest impurity importance") +
  theme_minimal()

```

```

oof <- map_dfr(1:5, function(k) {
  train <- df %>% filter(fold != k)
  test  <- df %>% filter(fold == k)
  m <- ranger(rf_form, data = train, num.trees = 500, seed = 42)
  test$pred <- predict(m, data = test)$predictions
  test
})

```

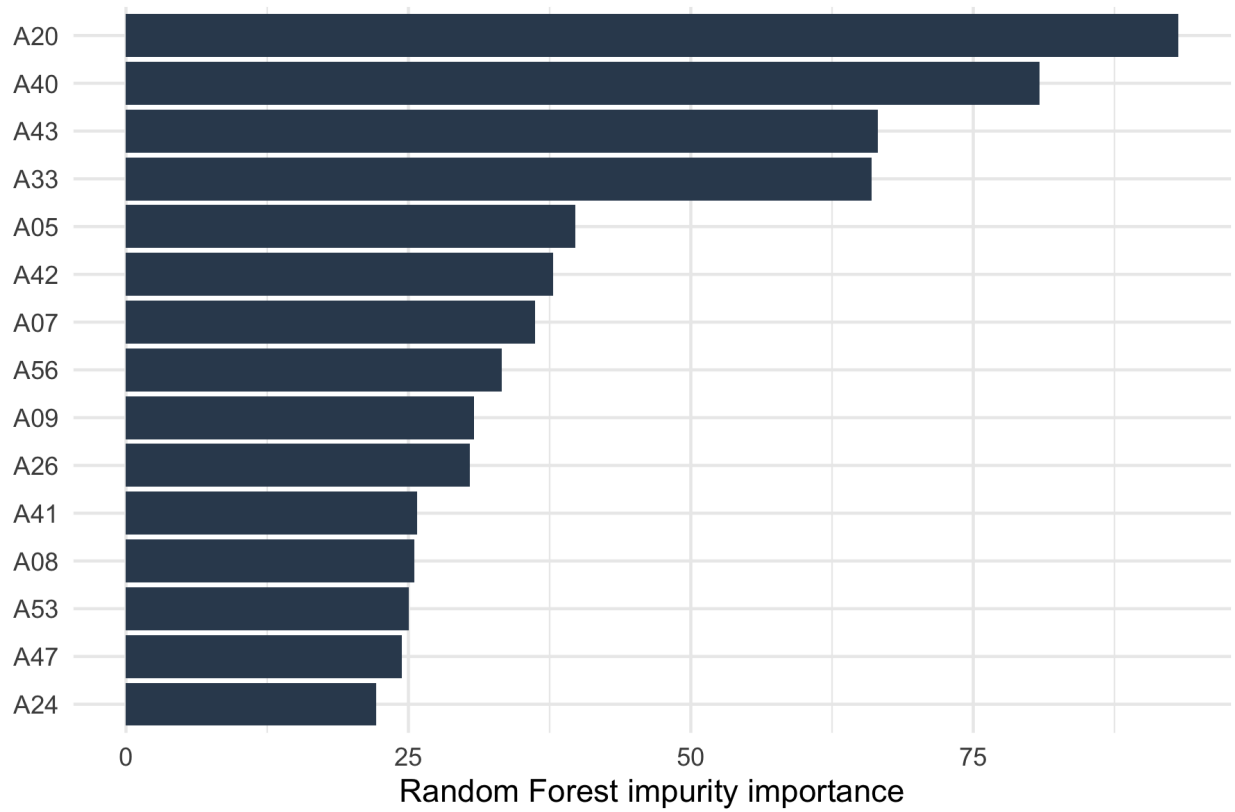


Figure 9: Top 15 AlphaEarth dimensions by Random Forest variable importance (impurity).


```
ggplot(oof, aes(log_soc, pred)) +
  geom_point(alpha = 0.3, size = 0.7, color = "#5b8def") +
  geom_abline(slope = 1, intercept = 0, color = "#c0392b", linetype = "dashed") +
  labs(x = "Observed log(1 + SOC)", y = "Predicted log(1 + SOC)") +
  theme_minimal()
```

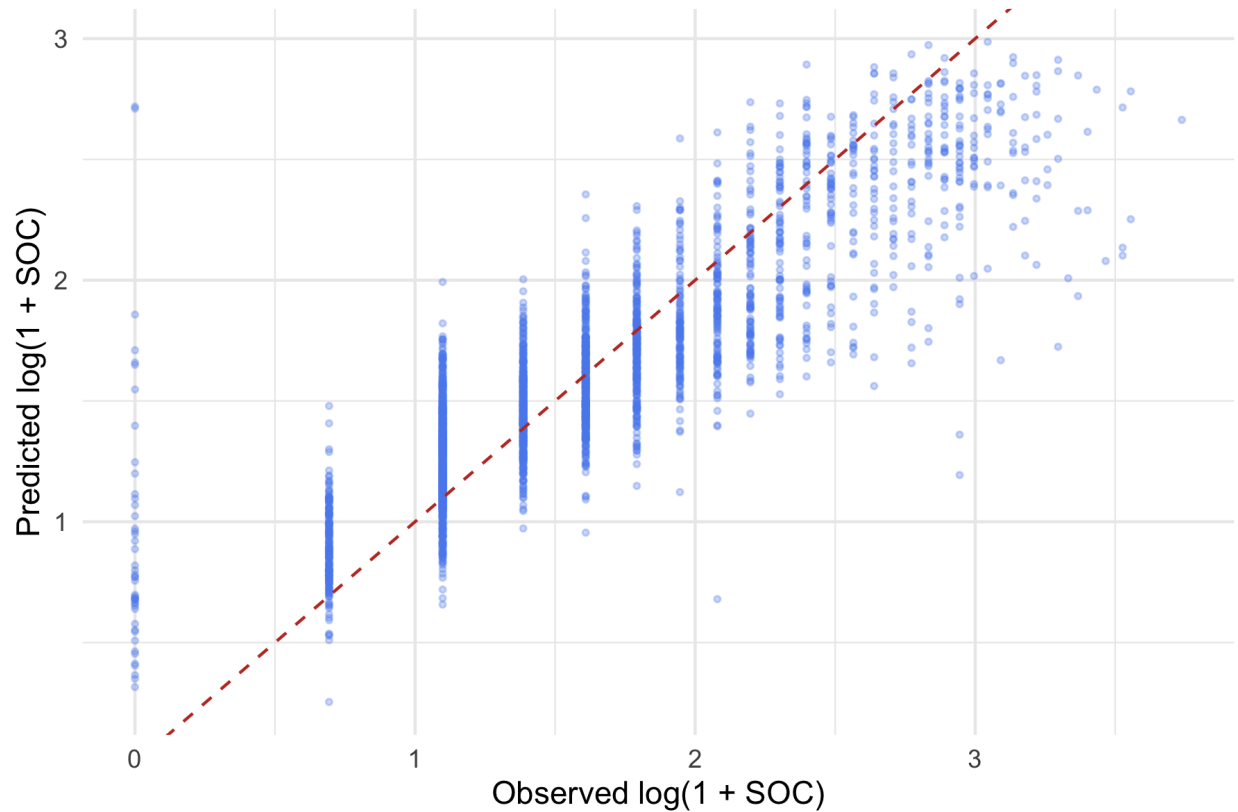


Figure 10: Out-of-fold predicted vs.~observed log-SOC across the 5 spatial folds.

6 Session info

```
sessionInfo()
```

```
## R version 4.4.2 (2024-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS 26.4.1
##
## Matrix products: default
## BLAS: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```

##
## time zone: America/Denver
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] ranger_0.18.0   car_3.1-3      carData_3.0-5  corrplot_0.95
## [5] boot_1.3-31     lubridate_1.9.4 forcats_1.0.0  stringr_1.5.1
## [9] dplyr_1.1.4     purrr_1.0.2    readr_2.1.5    tidyr_1.3.1
## [13] tibble_3.2.1    ggplot2_3.5.2  tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] utf8_1.2.4      generics_0.1.3  stringi_1.8.4   lattice_0.22-6
## [5] hms_1.1.3       digest_0.6.37   magrittr_2.0.3   evaluate_1.0.1
## [9] grid_4.4.2      timechange_0.3.0 RColorBrewer_1.1-3 fastmap_1.2.0
## [13] Matrix_1.7-1    Formula_1.2-5   gridExtra_2.3    mgcv_1.9-1
## [17] fansi_1.0.6     scales_1.4.0    abind_1.4-8      cli_3.6.5
## [21] rlang_1.1.6     splines_4.4.2   withr_3.0.2      yaml_2.3.10
## [25] tools_4.4.2     tzdb_0.5.0      vctrs_0.6.5      R6_2.5.1
## [29] lifecycle_1.0.4 pkgconfig_2.0.3 pillar_1.9.0     gtable_0.3.6
## [33] glue_1.8.0      Rcpp_1.0.13-1   xfun_0.49        tidyselect_1.2.1
## [37] knitr_1.49      farver_2.1.2    nlme_3.1-166     htmltools_0.5.8.1
## [41] labeling_0.4.3  rmarkdown_2.29  compiler_4.4.2

```